

week3 예습과제

≡ 다중 선택

week3

5장 합성곱 신경망 I

5.1 합성곱 신경망

딥러닝 역전파의 한계

: 순전파 과정에 따라 계산된 오차 정보가 신경망의 모든 노드(출력층 → 은닉층 → 입력층)로 전송

- 계산 과정 복잡
- 많은 자원(CPU, GPU, 메모리)을 요구
- 계산 시간 ↑

합성곱 신경망의 등장

: 앞의 문제를 해결하고자 한 신경망

- 이미지 전체를 한 번에 계산하는 것X
- 이미지의 국소적 부분을 계산 → 시간과 자원 절약 → 이미지의 세밀한 부분까지 분석 가능한 신경망

5.1.1 합성곱층의 필요성

- 이미지 분석
 - $n \times m$ 배열을 한 줄로 펼쳐서 각 픽셀에 가중치를 곱하여 은닉층으로 전달
 - 이미지를 펼치면 데이터의 공간적 구조를 무시한 채 분석하게 됨

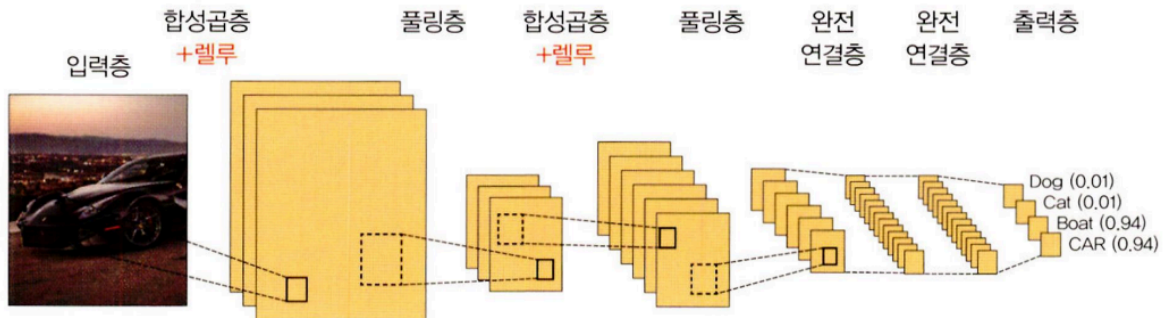
⇒ 이를 방지하기 위해 합성곱층 도입

5.1.2 합성곱 신경망 구조

합성곱 신경망 개요

- 음성 인식, 이미지/영상 인식에서 주로 사용되는 신경망
- 다차원 배열 처리에 특화
- 구조
 1. 입력층
 2. 합성곱층

3. 풀링층
4. 완전연결층
5. 출력층



1) 입력층

: 입력 이미지 데이터가 최초로 거치게 되는 계층

- 이미지 데이터: 3차원 데이터 (높이, 너비, 채널)
 - 채널
 - 그레이스케일(흑백) → 1값
 - 컬러 → 3값
 - ex - 높이: 4, 너비: 4, 채널: RGB인 이미지 → shape: (4,4,3)

2) 합성곱층

기본 개념

- **역할**: 입력 데이터에서 시각적 특징(선, 면, 질감 등)을 추출
- **커널 / 필터 (Kernel / Filter)**: 이미지 전체 영역을 훑으면서 특징을 계산해내는 행렬 (보통 3×3 , 5×5 크기 사용)
- **스트라이드 (Stride)**: 필터가 순회할 때 움직이는 간격(이동 단위)
- **특성 맵 (Feature Map)**: 필터를 거쳐 최종 추출된 결과물
 - **참고**: 필터 크기와 스트라이드 간격에 의해 특성 맵의 가로·세로 크기는 원본 입력보다 줄어들음

채널 별 필터 적용 구조

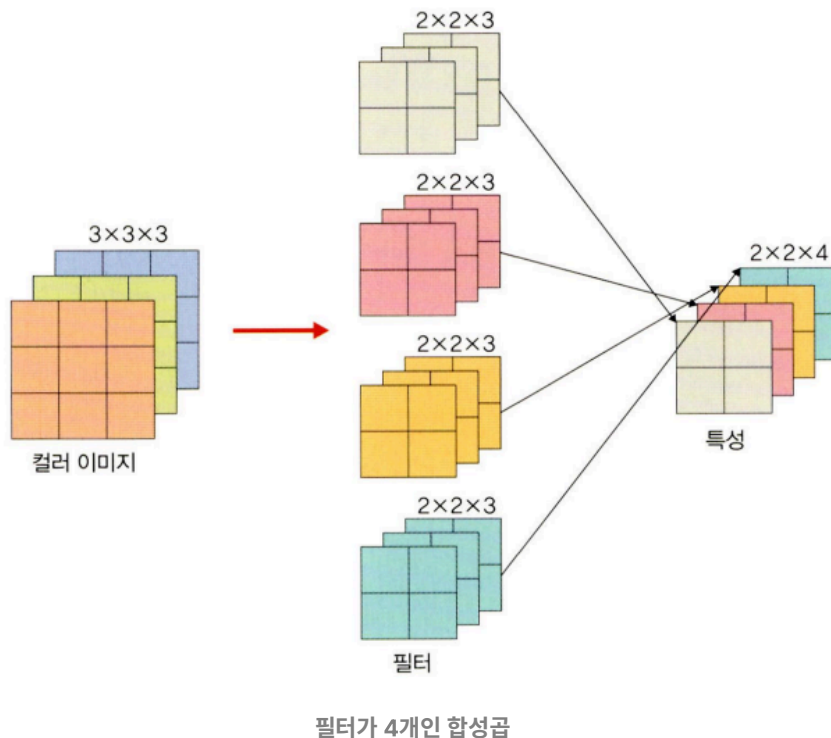
구분	입력 채널 수	필터의 채널 수	연산 특징
그레이스케일	1개 (흑백)	1개	단일 채널 필터로 단순 합성곱 수행
컬러 (RGB)	3개	3개	필터의 채널 수는 입력의 채널 수와 무조건 일치함

- **컬러 이미지 합성곱의 핵심**:
 - 필터의 형상이 ($2 \times 2 \times 3$)이라면, 이는 **3채널짜리 3차원 필터 1개**를 의미 (필터가 3개가 아님)

- RGB 각 채널에 대응되는 서로 다른 가중치로 3번의 합성곱을 계산한 뒤, **결과값 3개를 모두 합산하여 단 1개의 2차원 출력**을 만듦

필터 개수와 특성 맵(Feature Map) 채널의 관계

- **필터 1개의 역할:**
 - 입력 데이터의 모든 채널(RGB 등)을 하나로 통합 계산하여 **1개 채널의 특성 맵**을 생성
- **출력 특성 맵의 채널 수:**



- 입력 채널 수와 무관하게, '**사용한 필터의 개수**'에 따라 출력 특성 맵의 채널 수가 결정 됨
- 예시: 3채널(RGB) 입력에 **필터 16개**를 적용 → **16채널** 특성 맵 생성 (필터 1개당 1채널 담당)

💡 핵심 요약

- 입력 데이터 채널 수 = 필터 1개의 채널 수
- 적용한 필터 개수 = 최종 특성 맵(Feature Map)의 채널 수

▼ 그레이스케일 이미지의 합성곱 예

1. 입력 이미지에 3x3 필터 적용



입력 이미지와 필터를 포개놓고 대응되는 숫자끼리 곱한 후 모두 더함
 $\rightarrow 1*1 + 0*0 + 1*0 + 0*0 + 1*1 + 0*0 + 0*1 + 0*0 + 1*1 = 3$

2. 필터가 1(스트라이드 값)만큼 이동하여 1번을 반복



필터가 1만큼 이동

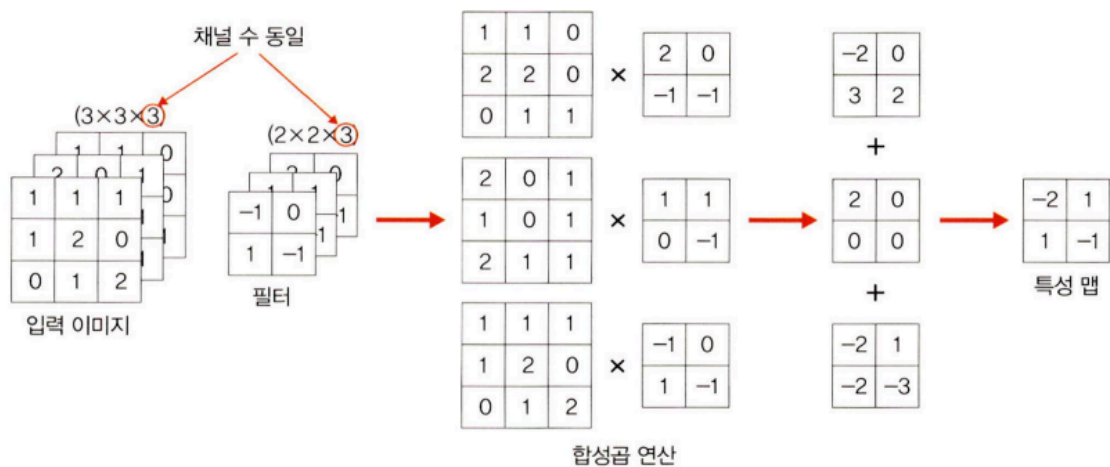
3. 마지막 끝까지 칸을 훑으며 이동 → 특성 맵을 꽉 채우면 종료



필터가 1만큼 마지막으로 이동

▼ 컬러 이미지의 합성곱

- 채널수를 제외하곤 그레이스케일의 합성곱 과정과 동일
- 각 채널에 대한 결과값을 모두 합산하여 하나의 특성맵으로 출력함



합성곱층 요약

- 입력 데이터: $(W_1 \times H_1 \times D_1)$ (W_1 : 가로, H_1 : 세로, D_1 : 채널 또는 깊이)
- 하이퍼파라미터
 - 필터 개수: K
 - 필터 크기: F
 - 스트라이드: S
 - 패딩: P (Padding)
- 출력 데이터 (Output Data): (W_2, H_2, D_2)
 - $W_2 = (W_1 - F + 2P)/S + 1$
 - $H_2 = (H_1 - F + 2P)/S + 1$
 - $D_2 = K$



패딩

: 합성곱 전에 입력의 가장자리에 값을 추가하는 것

- 가장 흔하게는 0을 추가
 - 원래 입력: $[1, 1, 1, 1, 1]$ —패딩→ $[0, 1, 1, 1, 1, 1, 0]$
 - $[0, 1, 1]$ → 계산**
 - $[1, 1, 1] \rightarrow$ 계산
 - $[1, 1, 1] \rightarrow$ 계산
 - $[1, 1, 1] \rightarrow$ 계산
 - $[1, 1, 0]$ → 계산**
 - ⇒ 입력 5개 → 출력 5개 (패딩 X면 출력 3개)
- 합성곱 결과의 출력 크기를 조절할 수 있으며, 입력과 출력의 크기를 같게 유지할 때 사용
- 가장자리의 정보가 합성곱에 충분히 활용되는 효과도 있음

2) 풀링층

- 역할: 특성 맵의 차원을 다운 샘플링 → 연산량을 감소시킴 + 주요 특성 벡터 추출 ⇒ 효과적 학습



다운 샘플링(sub-sampling)

: 이미지를 축소하는 것



풀링 연산

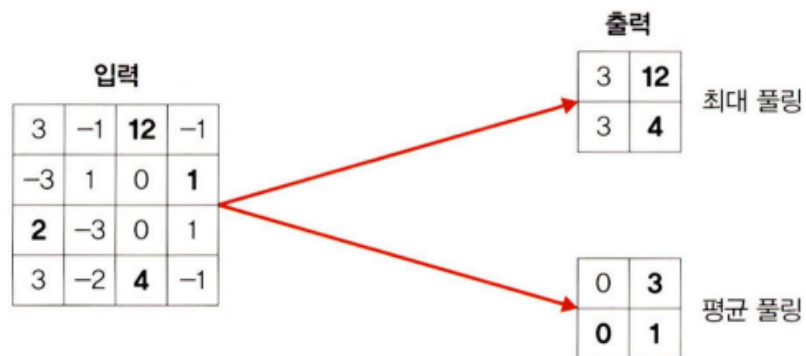
- 평균 풀링(average pooling):** 대상 영역에서 평균을 반환
 - 특징: 각 커널 값을 평균화시켜 중요한 가중치를 갖는 값의 특성이 희미해질 수 있음
→ 평균 풀링보다는 **최대 풀링 사용** ↑
 - 과정
 - 각 커널 안에 들어오는 값들의 평균값을 출력
 - 스트라이드 값만큼 건너뛰며 1을 반복하고, 끝까지 훑으면 끝남
- 최대 풀링(max pooling):** 대상 영역에서 최대값을 추출
 - 과정
 - 각 커널 안에 들어오는 값 중 최대값을 출력



- 스트라이드 값만큼 건너뛰며 1을 반복하고, 끝까지 훑으면 끝남



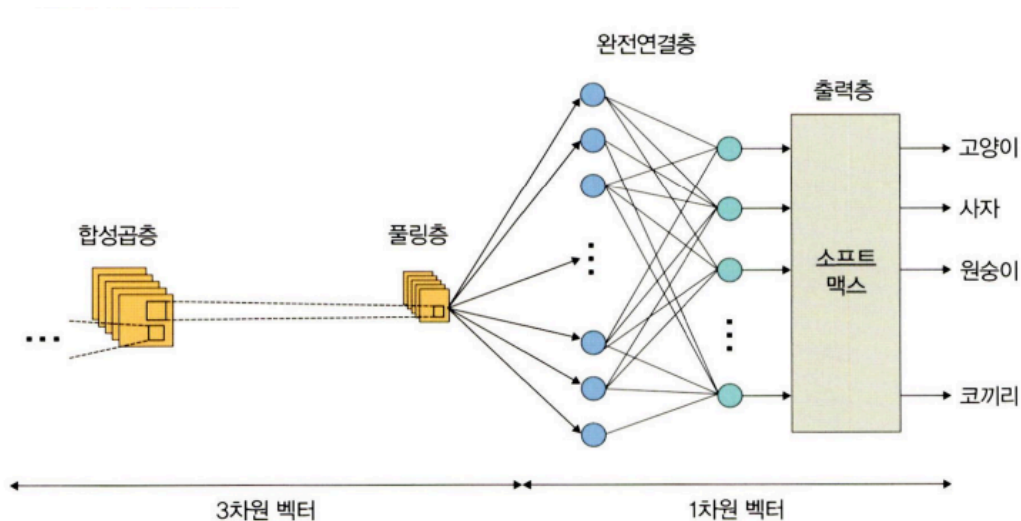
- 출력값 비교



풀링층 요약

- 입력 데이터: $(W_1 \times H_1 \times D_1)$ (특성 맵)
- 하이퍼파라미터
 - 필터 크기: F
 - 스트라이드: S
- 출력 데이터 (Output Data): (W_2, H_2, D_2)
 - $W_2 = (W_1 - F) / S + 1$
 - $H_2 = (H_1 - F) / S + 1$
 - $D_2 = D_1$

3) 완전연결층(fully connected layer)



- 합성곱층과 풀링층을 거치면서 차원이 축소된 특성 맵은 최종적으로 완전연결층으로 전달됨
- 3차원 벡터이던 이미지를 1차원 벡터로 펼치게(flatten) 됨

4) 출력층(output layer)

- **소프트맥스 함수**를 활성화 함수로 사용
 - 소프트맥스 함수의 출력값
 - 입력받은 값을 0~1 사이의 값
 - 이미지가 각 레이블에 속할 확률 값
- **최종값:** 가장 높은 확률 값을 갖는 레이블

5.1.3 1D, 2D, 3D 합성곱

→ 필터가 이동하는 축의 수에 따라 1D, 2D, 3D 합성곱으로 구분

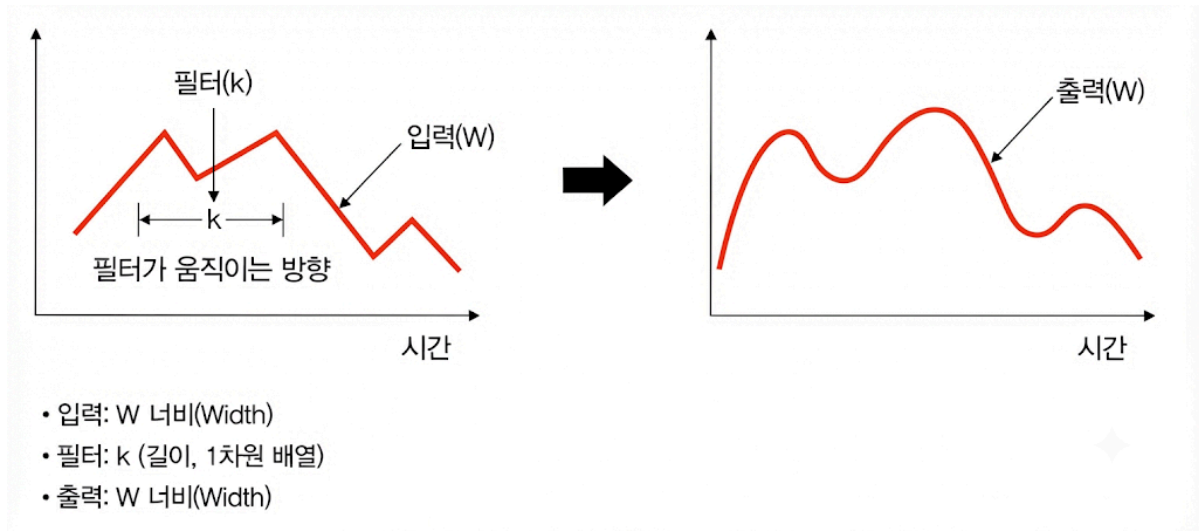
구분	입력 크기	필터 크기	필터 이동 축	출력 형태
1D	W	k	1개 축	1차원
2D	(W, H)	(k, k)	2개 축	2차원
3D	(W, H, L)	(k, k, d)	3개 축	3차원
3D 입력을 갖는 2D	(W, H, L)	(k, k, L)	2개 축	2차원

1) 1D 합성곱

- 필터가 하나의 축(시간축 등)을 따라 이동하며 연산
 - 입력: W
 - 필터: k
 - 출력: $W \rightarrow$ 패딩을 사용하는 경우
 - 출력 형태: 1차원 배열
- 그래프의 곡선 완화, 시계열 데이터 등에 활용

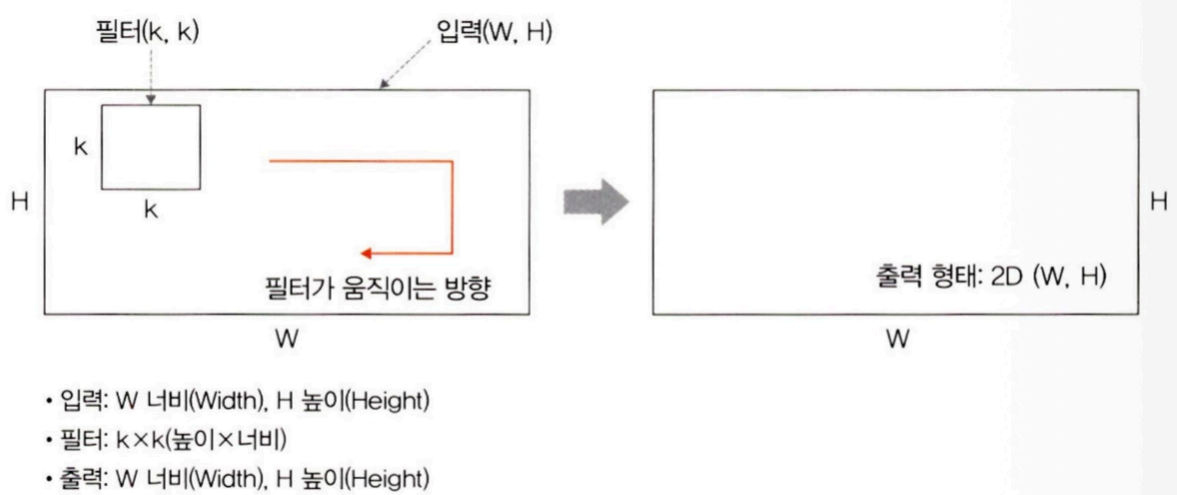
- 예시

- 입력 = $[1, 1, 1, 1, 1]$ → $W = 5$
- 필터 = $[0.25, 0.5, 0.25]$ → $k = 3$
- 패딩 없이 연산 → 출력 = $[1, 1, 1]$



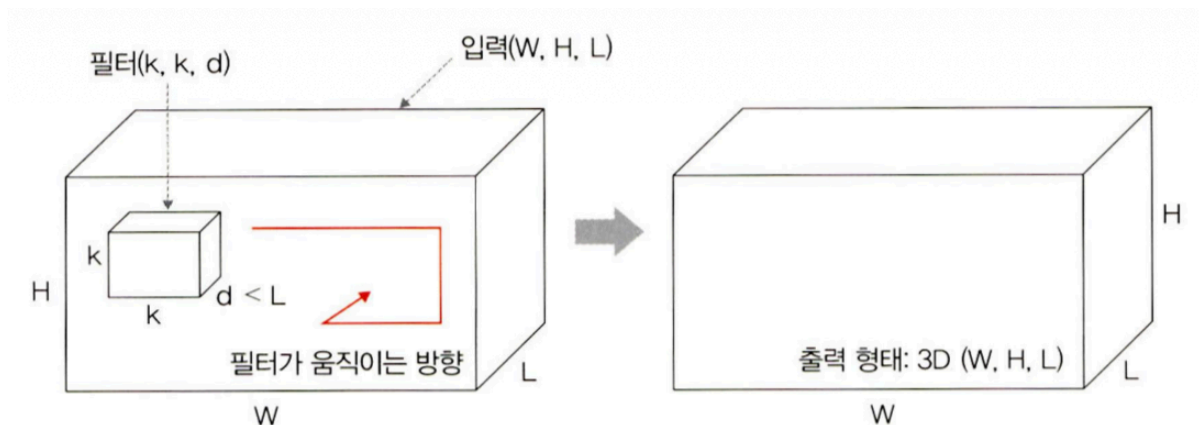
2) 2D 합성곱

- 필터가 가로축과 세로축을 따라 이동하며 연산
 - 입력: $W \times H$
 - 필터: $k \times k$
 - 출력: $W \times H$ → 패딩을 사용하는 경우
 - 출력 형태: 2차원 행렬
- 이미지와 같은 2차원 데이터에 주로 활용



3) 3D 합성곱

- 필터가 가로·세로·깊이의 세 축을 따라 이동하며 연산
 - 입력: $W \times H \times L$
 - 필터: $k \times k \times d$
 - 출력: $W \times H \times L \rightarrow$ 패딩을 사용하는 경우
 - 출력 형태: 3차원 데이터
- $d < L$ 이면 필터가 깊이 방향으로도 이동하며 3차원 특징을 추출
- $d = L$ 이면 깊이 방향으로 이동하지 않으므로 3D 입력을 갖는 2D 합성곱이 됨



- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: $k \times k$ (높이 $\times$ 너비), d: 깊이(depth)
- 출력: W 너비(Width), H 높이(Height), L 길이(Length)

4) 3D 입력을 갖는 2D 합성곱

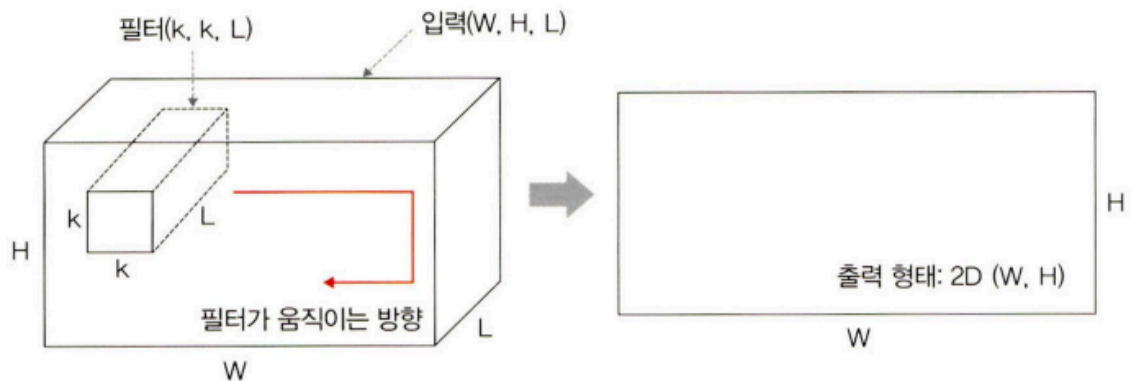
- 입력은 3차원이지만 필터의 깊이가 입력의 깊이와 같아, 필터가 깊이 방향으로 이동하지 않고 가로·세로 방향으로만 이동
 - 입력: (W, H, L)
 - 필터: (k, k, L)
 - 출력: (W, H)
 - 출력 형태: 2차원 행렬

입력의 깊이 $L =$ 필터의 깊이 L

- $\rightarrow$ 깊이 방향으로 필터를 이동할 수 없음
- $\rightarrow$ 가로·세로 2개 축으로만 이동
- $\rightarrow$ 출력은 2차원

- 예시
 - 입력 $(224, 224, 3)$
 - 필터 $(k, k, 3)$
 - 출력 $(224, 224)$ (패딩 사용 시)

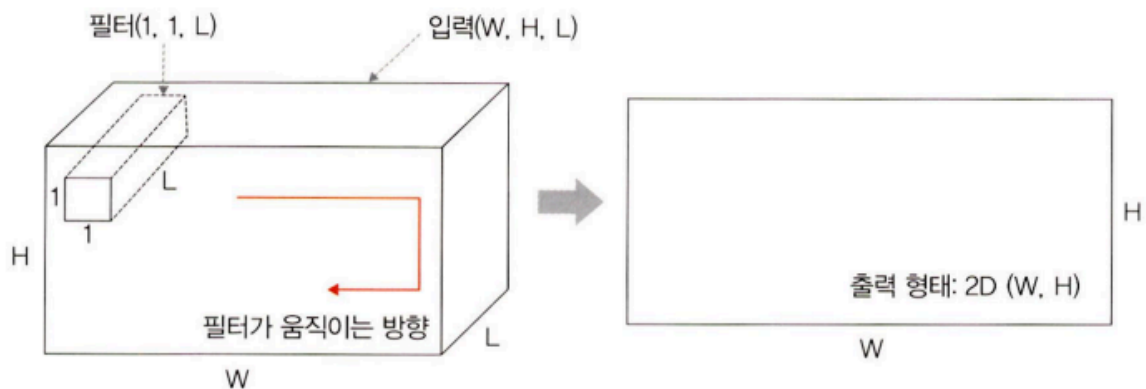
- RGB 이미지처럼 **채널을 포함한 3차원 입력**에 적용할 수 있음
- 대표적인 예: **LeNet-5, VGG**



- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: $k \times k$ (높이 $\times$ 너비), L 길이(Length)
- 출력: W 너비(Width), H 높이(Height)

5) 1×1 합성곱

- 3D 입력에 적용되는 특수한 형태의 2D 합성곱
 - 입력: (W, H, L)
 - 필터: (1, 1, L)
 - 출력: (W, H)
- 필터의 공간적 크기가 1×1 이므로 각 위치의 채널 방향 정보만 결합
 - 여러 개의 1×1 필터를 사용하면 채널 수를 조절할 수 있음
 - 채널 수를 줄이면 연산량을 감소시킬 수 있음
- 대표적인 예: **GoogLeNet**



- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: 1×1 (높이 $\times$ 너비), L 길이(Length)
- 출력: W 너비(Width), H 높이(Height)

5.2 합성곱 신경망 맛보기



fashion_mnist 데이터 셋

- torchvision에 내장된 예제 데이터
- 훈련 데이터(train_images): 0에서 255 사이의 값을 갖는 28*28 크기의 넘파이 배열
 - 운동화, 셔츠, 샌들 등의 작은 이미지의 모음
 - 열 가지로 분류될 수 있는 28*28 픽셀의 이미지 7만 개로 구성
- 레이블 데이터(train_labels): 0에서 9까지의 정수(운동화, 셔츠 등의 클래스) 값을 갖는 배열
 - 0 : T-Shirt
 - 1 : Trouser
 - 2 : Pullover
 - 3 : Dress
 - 4 : Coat
 - 5 : Sandal
 - 6 : Shirt
 - 7 : Sneaker
 - 8 : Bag
 - 9 : Ankle Boot



GPU 설정

1. 단일 GPU(1개)를 쓸 때
→ 컴퓨터에 GPU가 하나만 있거나, 하나의 GPU만 지정해서 모델을 학습시킬 때 쓰는 가장 기본적인 코드
2. 멀티 GPU(여러 개)를 쓸 때 (`nn.DataParallel`)
 - a. 만약 컴퓨터에 그래픽카드가 2개 이상 꽂혀 있다면, 연산을 한 곳에만 몰아주지 않고 **여러 GPU에 나누어 동시에 처리**(`DataParallel`)하게 만들어 속도를 훨씬 빠르게 윈도우/리눅스 환경에서 가속할 때 사용

```
device = torch.device("cuda:0" if torch.cuda.is_available()
else "cpu")
model = Net()
model.to(device)
```

```
device = torch.device("cuda" if torch.cuda.is_available() e
lse "cpu")
model = Net()
```

```
if torch.cuda.device_count() > 1:
    model = nn.DataParallel(model) # 오타 수정: net -> model
    model.to(device)
```

코랩은 기본적으로 GPU가 '1개'입니다

- 구글 코랩은 무료 버전이든 유료 버전이든 기본적으로 사용자에게 **그래픽카드(GPU)**를 딱 1개만 할당
- 따라서 두 번째 내용인 **여러 개의 GPU**를 쓰는 코드(`nn.DataParallel`)는 코랩 환경에서는 쓸 일이 거의 없음 (컴퓨터 본체에 그래픽카드를 여러 개 꽂아둔 개인 서버나 연구실 컴퓨터용 기능)



`transforms` VS. `transform`

1. `transforms` (복수형, s가 붙음)
 - **용도:** 모듈이나 패키지의 이름 (`import torchvision.transforms as transforms`)
 - **이유:** 이미지 크기를 바꾸고(`Resize`), 텐서로 바꾸고(`ToTensor`), 정규화하는(`Normalize`) 등 여러 가지 이미지 변환 기능들이 모여 있는 '종합 선물 세트(모듈)'이기 때문에 여러 개를 뜻하는 복수형 `transforms` 라는 이름을 씀
2. `transform` (단수형)
 - **용도:** 데이터셋(Dataset)에 인자로 전달할 때 (`Dataset(..., transform=...)`)
 - **이유:** 데이터셋 입장에서 보면, "이 데이터를 로드할 때 적용할 '변환 파이프라인(또는 묶음)' 하나"를 지정해 주는 것이기 때문에 단수형 `transform` 을 씀



왜 입력부터 Dropout하지 않을까?

- **입력(Input) 단계에서 Dropout을 적용하면**
 - FashionMNIST와 같은 이미지에서는 각 픽셀이 모여 **옷의 윤곽, 소매 모양 등의 구체적인 형태**를 구성함
 - 입력 픽셀을 무작위로 일부 제거하면 **이미지의 중요한 공간 정보가 손실**되어 학습이 어려워질 수 있음
- **중간 레이어에서 Dropout을 적용하는 이유**
 - `fc1` 등을 거친 데이터는 단순한 픽셀이 아니라 **선, 패턴 등의 추상적인 특징(Feature)**을 나타냄
 - 이 단계에서 일부 뉴런을 무작위로 제거하면 특정 특징이나 뉴런에 **과도하게 의존하는 것을 방지**할 수 있음

- 결과적으로 다양한 특징을 활용하도록 학습하여 **과적합을 완화**할 수 있음

정리: 입력 단계에서는 원본 이미지의 정보 손실을 줄이고, 중간 레이어에서는 일부 뉴런을 제거하여 특정 특징에 대한 의존을 줄이는 것이 목적



객체 지향 프로그래밍 (OOP)

- 파이토치는 C++ 기반이므로 **객체 지향 프로그래밍(OOP)**의 특징을 사용함
- **클래스:** 객체를 만들기 위한 설계도(틀)
- **객체:** 클래스를 기반으로 생성되어 실제로 사용되는 데이터
- 객체 지향 프로그래밍을 사용하는 주요 이유 → **재사용성**
- `객체명 = 클래스명()` → 클래스를 이용해 객체를 생성



클래스와 함수

- **함수:** 특정 작업을 수행하는 코드 단위
 - 함수 실행 결과를 계속 사용하려면 **별도의 변수에 저장**해야 함

```
def add(num1, num2):
    result = num1 + num2
    return result
```

- **클래스:** 함수뿐만 아니라 **관련된 데이터(변수)까지 하나로 묶어 관리**하고 재사용할 수 있음
 - 클래스로 생성한 **각 객체**는 **서로 독립적인 데이터**를 가짐

```
class Calc:
    def __init__(self):
        self.result = 0

    def add(self, num1, num2):
        self.result = num1 + num2
        return self.result

obj1 = Calc()
obj2 = Calc()
```

- `obj1`, `obj2` 는 각각 **독립적인 result** 를 가짐
- **관련된 데이터와 함수를 함께 관리**할 수 있음



PyTorch 순전파(Forward Pass)

- **forward()** 실행 방식
 - `forward()` 를 직접 호출하지 않고 `model(input_data)` 형태로 모델을 호출
 - PyTorch가 내부적으로 `forward()` 를 실행함
- 순전파(Forward Pass)
 - 입력값 `x`를 모델에 넣어 예측값 `y`를 계산하는 전체 과정
 - 가중치·편향 계산, 활성화 함수 적용 등의 과정을 거침
 - $H(x)$: 모델이 입력 `x`를 받아 예측값을 계산하는 함수
 - 모델 구조와 활성화 함수에 따라 형태가 달라짐



`view()` VS `reshape()`

- 공통점: 둘 다 텐서의 Shape을 변경하는 함수
- 차이점

	<code>view()</code>	<code>reshape()</code>
메모리	기존 텐서와 공유	가능하면 공유, 필요하면 복사
Contiguous	필요	필요 없음
비연속 텐서	에러 발생 가능	알아서 처리

- **view()**
 - 기존 메모리를 그대로 사용하여 Shape만 변경
 - 텐서가 **Contiguous**해야 함
 - `transpose()`, `permute()` 등으로 비연속 텐서가 된 경우 사용에 주의
- **reshape()**
 - 가능하면 `view()` 처럼 메모리를 공유
 - 메모리 공유가 불가능하면 새로운 텐서를 만들어 복사
 - 따라서 비연속 텐서에도 사용 가능

정리: `view()` 는 **Contiguous** 조건이 필요한 대신 메모리 복사가 없고, `reshape()` 는 필요하면 복사하여 더 유연하게 Shape을 변경함.



`nn.xx` VS `nn.functional.xx`

둘 다 같은 연산을 수행할 수 있지만, **형태와 사용 위치, 파라미터 관리 방식**이 다름.

⇒ **Module로 등록해서 관리하느냐 vs 연산 함수를 그때그때 호출하느냐**

1. `nn.xx` → **Module(레이어)**로 사용

- 클래스 형태 → 객체를 만들어 사용
- 모델의 레이어를 정의하는 부분에서 사용
- Weight/Bias가 있는 경우 파라미터를 자동으로 생성·관리
- `nn.Sequential`에 넣을 수 있음

```
self.conv = nn.Conv2d(3, 64, 3, padding=1)
self.relu = nn.ReLU()

def forward(self, x):
    x = self.conv(x)
    x = self.relu(x)
    return x
```

→ `__init__()`에서 레이어를 정의하고, `forward()`에서 사용

2. `F.xx` → **함수로 직접 호출**

- 함수 형태
- 주로 `forward()`에서 필요한 연산을 직접 호출
- Weight/Bias가 필요한 연산은 직접 전달
- `nn.Sequential`에 넣을 수 없음

```
def forward(self, x):
    x = F.conv2d(x, weight, bias, padding=1)
    x = F.relu(x)
    return x
```

→ 필요한 위치에서 바로 함수 호출

▼ `nn.Sequential`

여러 Module을 순서대로 연결하는 기능

```
model = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 10)
)
```

→ `Linear` → `ReLU` → `Linear` 순서로 연산

- 핵심 비교

	<code>nn.xx</code>	<code>F.xx</code>
형태	클래스 → Module	함수
사용 위치	레이어 정의 후 사용	필요한 곳에서 직접 호출
파라미터 관리	Module이 관리	직접 관리
<code>Sequential</code>	O	X
일반적인 레이어 구성	주로 사용	필요할 때 사용



배치 단위 학습

- 데이터를 하나씩 처리하는 것이 아니라, 여러 개를 하나의 배치(batch)로 묶어 한 번에 처리
 - 예: 배치 크기 `100`
 - 입력: `[100, 784]`
 - 출력: `[100, 10]`
 - → 100개의 데이터에 대한 예측값을 한 번에 계산
 - 학습 과정
 1. 순전파: 배치 단위로 예측값 계산
 2. Loss 계산: 각 데이터의 오차를 구한 뒤 보통 평균 Loss 계산
 3. 역전파: `loss.backward()` 로 배치의 오차를 바탕으로 gradient 계산
 4. 가중치 업데이트: `optimizer.step()` 으로 가중치를 한 번 업데이트
- 정리: 배치 단위로 순전파 → Loss 계산 → 역전파 → 가중치 1회 업데이트
- 장점
 - 여러 데이터를 묶어 병렬 연산할 수 있어 효율적
 - 여러 데이터의 gradient를 함께 반영하여 학습이 안정적



정확도(Accuracy) 계산과 주의점

- 정확도(Accuracy)
 - 모델이 전체 데이터 중 얼마나 많이 맞췄는지 나타내는 지표
 - $Accuracy = \frac{\text{맞춘 개수}}{\text{전체 개수}} \times 100$

- 오인식률(Error Rate)
 - 전체 데이터 중 틀린 비율
 - $Error\ Rate = \frac{\text{틀린 개수}}{\text{전체 개수}} \times 100$
 - $Error\ Rate = 1 - Accuracy$

정확도만으로 판단하면 안 되는 경우

불균형 데이터(Imbalanced Data)

→ 각 클래스의 데이터 수가 크게 차이 나는 경우

- ex
 - 정상인은 99명, 환자는 1명인 데이터에서 모델이 모든 사람을 정상이라고 예측
 - ⇒ 맞춘 사람: 99명, 정확도: 99%
 - ⇒ but 환자를 한 명도 찾아내지 못했기 때문에, 정확도 99%만 보고 좋은 모델이라고 판단하기는 어려움
- 데이터가 한쪽 클래스에 치우쳐 있다면, 정확도만으로 모델의 성능을 판단하지 않도록 주의해야 함.



CNN의 합성곱 → 풀링 과정

1. **Conv2d** — 합성곱 연산
필터가 입력 이미지를 이동하면서 각 영역에 대해 곱하고 더하는 연산을 수행한다.
→ 연산 결과가 모여 **특성 맵(Feature Map)**을 만든다.
 2. **MaxPool2d** — 최대 풀링
Conv2d를 통해 만들어진 **특성 맵**에 풀링을 적용한다.
 - 일정한 크기의 윈도우가 특성 맵을 이동
 - 각 영역에서 **가장 큰 값(Maximum)**을 선택
 - 특성 맵의 크기를 줄이면서 중요한 특징을 유지
- 전체 흐름
입력 이미지 → **Conv2d** → 특성 맵 → **MaxPool2d** → 축소된 특성 맵
- ▮ **Conv**는 특징을 찾아내고, **MaxPool**은 그중 중요한 값만 남겨 크기를 줄임



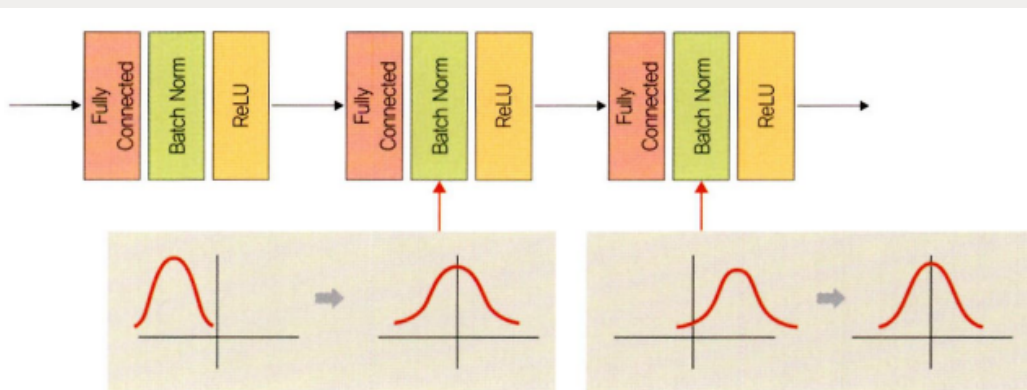
```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size, padding=1)
```

- **in_channels** : 입력 채널 수(깊이)

- 흑백 이미지는 1, RGB 이미지는 3인 경우가 많음
- `out_channels` : 출력 채널 수(필터 수)
- `kernel_size`
 - 이미지 특징을 찾아내기 위한 공용 파라미터 → CNN에서의 학습 대상은 필터 파라미터
 - 정수로 지정 → 정사각형 커널
 - (n, m)으로 지정 → 직사각형 커널
- `padding` : 패딩의 크기
 - 출력 크기 조정 → 패딩 값이 클수록 출력 크기도 커짐
 - 입력 데이터 주위에 0을 채움



BatchNorm2d (배치 정규화)



1. BatchNorm2d 란?

- `BatchNorm2d` 는 CNN에서 합성곱 연산 후 데이터의 분포를 안정적으로 만들어 주는 **정규화 레이어**
- 미니배치 단위로 평균과 분산을 계산
- **채널별로 독립적으로 정규화**
- 정규화한 뒤 학습 가능한 γ (scale)와 β (shift)를 이용해 값을 다시 조정
 - 단순히 모든 값을 평균 0, 표준편차 1로 고정하는 것이 아니라, **정규화 후 학습을 통해 적절한 값으로 조정할 수 있음**

2. 왜 사용하는가?

- **학습 안정화**: 각 층의 값의 분포를 안정적으로 유지
- **학습 속도 향상**: 학습이 더 빠르고 안정적으로 진행될 수 있음
- **과적합 방지 보조**: 미니배치 통계의 변동으로 인해 약간의 규제 효과가 발생

3. 어떻게 정규화하는가?

- ex
 - Batch Size = 100, Channel = 32인 경우,
 - 1번 채널의 값만 모아서 평균과 분산을 계산하고,
 - 2번 채널은 2번 채널끼리 따로 계산한다.
- 채널별로 따로 계산하지만, 같은 채널에서는 배치의 여러 이미지에 있는 값을 함께 사용

4. CNN에서의 위치

- Conv2d → BatchNorm2d → ReLU → MaxPool2d
 - Conv2d : 특징 추출
 - BatchNorm2d : 데이터 분포 안정화
 - ReLU : 활성화 함수 적용
 - MaxPool2d : 중요한 값만 남기며 크기 축소



MaxPool2d

- 특성 맵의 공간적 크기(가로·세로)를 줄이는 데 사용
- `nn.MaxPool2d(kernel_size=2, stride=2)`
 - `kernel_size` : 풀링할 영역의 크기
 - `stride` : 풀링 윈도우가 이동하는 간격
 - MaxPool2d : 각 영역에서 최댓값을 선택
 - 일반적으로 `kernel_size=2, stride=2` 를 사용하면 가로·세로 크기가 절반으로 줄어듦



출력 크기 구하기

- Conv2d 출력 크기
 - 식

$$Output = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$$

- W : 입력 크기
- F : 커널 크기
- P : 패딩

- S : 스트라이드

◦ ex

```
nn.Conv2d(
    in_channels=1,
    out_channels=32,
    kernel_size=3,
    padding=1
)
```

- FashionMNIST의 입력 이미지가 28×28 이고 $\text{stride}=1$ 이라면,
 $28 - 3 + (2 \times 1)1 + 1 = 28 \frac{28-3+(2 \times 1)}{1} + 1 = 28$

입력: [N, 1, 28, 28]
 출력: [N, 32, 28, 28]

- 28×28 : padding 때문에 공간 크기 유지
- $1 \rightarrow 32$: $\text{out_channels}=32$ 이므로 채널 수가 32개가 됨

• MaxPool2d 출력 크기

◦ 식

$$Output = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$$

- W : 입력 크기
- F : 커널 크기 (kernel_size)
- P : 패딩
- S : 스트라이드 (stride)

◦ ex

```
nn.MaxPool2d(kernel_size=2, stride=2)
```

- 입력이 28×28 이라면
 $28 \div 2 = 14, 28 \div 2 = 14$

입력: [N, 32, 28, 28]
 출력: [N, 32, 14, 14]

- `MaxPool2d` 는 채널 수는 그대로 유지하고, 주로 가로·세로 크기를 줄인다.



4. CNN → 완전연결층

`Conv2d`, `MaxPool2d` 는 이미지 형태의 데이터를 처리
but `Linear` 는 1차원 형태로 펼친 데이터를 입력으로 받음
⇒ 완전연결층으로 전달하기 전에 `Flatten` 이 필요하다.

```
out = out.view(out.size(0), -1)
```

- `out.size(0)` : 배치 크기
- `1` : 나머지 차원은 자동으로 계산



전체 흐름

```
이미지
↓
Conv2d
↓
특성 맵
↓
MaxPool2d
↓
크기 축소
↓
Flatten
↓
Linear
↓
분류
```

정리: `Conv2d` 와 `MaxPool2d` 를 거치면서 특성 맵의 크기와 채널 수가 결정되고, 마지막에는 이를 1차원으로 펼쳐 `Linear` 에 전달